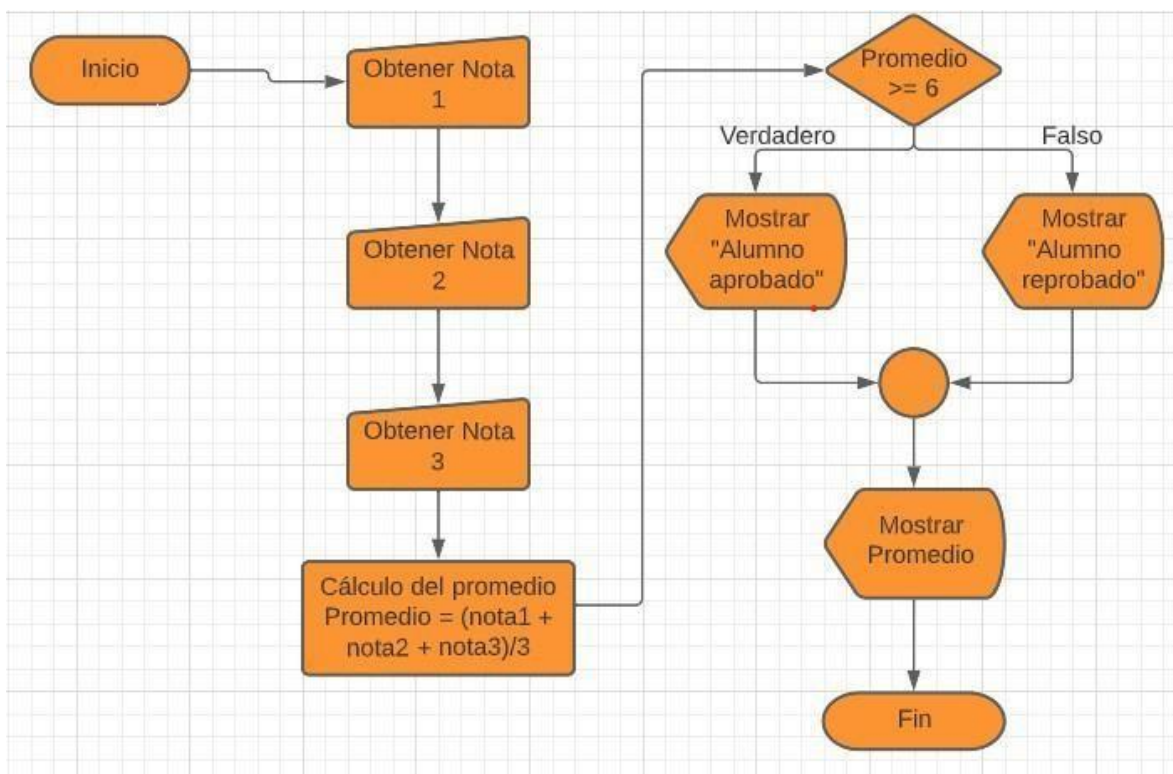




Universidad de El Salvador

Facultad Multidisciplinaria de Occidente - Ingeniería en Desarrollo de Software

Asignatura:
Manejo de Estructura de
Datos

Unidad #: 2
Semana 10
Fecha: 28-04-2025

Temas: Fundamentos de Programación y Análisis de
Algoritmos
Autor: _____

Introducción

Bienvenidos al material de lectura de la semana #10 de la Unidad #2 de la materia Manejo de Estructura de Datos. En este material explicaremos los fundamentos de programación y análisis de algoritmos.

Este material de lectura está diseñado para proporcionar una base sólida sobre los fundamentos de programación y análisis de algoritmos, preparándote para enfrentar los desafíos y oportunidades que encontrarás en tu carrera profesional. A medida que avances en este material, te alentamos a participar activamente de las actividades, para favorecer tu proceso de aprendizaje.

¡Antes de comenzar, asegúrate de tener papel y lápiz a mano para las actividades interactivas!

Sección 1: Eficiencia de los algoritmos

Medida del uso de los recursos computacionales requeridos por la ejecución de un algoritmo en función del tamaño de las entradas.

$T(n)$: Tiempo empleado para ejecutar el algoritmo con una entrada de tamaño n .

El análisis de algoritmos es una parte importante de la Teoría de Complejidad computacional más amplia, que provee estimaciones teóricas para los recursos que necesita cualquier algoritmo que resuelva un problema computacional dado. Estas estimaciones resultan ser bastante útiles en la búsqueda de algoritmos eficientes.

A la hora de realizar un análisis teórico de algoritmos es común calcular su complejidad en un sentido asintótico, es decir, para un tamaño de entrada suficientemente grande. La Cota superior asintótica y las notaciones omega (cota inferior) y theta (caso promedio) se usan con esa finalidad. Por ejemplo, decimos que la búsqueda binaria se ejecuta en una cantidad de pasos proporcional a un logaritmo, en $O(\log(n))$, coloquialmente «en tiempo logarítmico».

Normalmente, las estimaciones asintóticas se utilizan porque diferentes implementaciones del mismo algoritmo no tienen por qué tener la misma eficiencia. No obstante, la eficiencia de dos implementaciones «razonables» cualesquiera de un algoritmo dado están relacionadas por una constante multiplicativa llamada constante oculta.

La medida exacta (no asintótica) de la eficiencia a veces puede ser computada, pero para ello suele hacer falta aceptar supuestos acerca de la implementación concreta del algoritmo, llamada **modelo de computación**.

Un modelo de computación puede definirse en términos de un ordenador abstracto, como la Máquina de Turing, y/o postulando que ciertas operaciones se ejecutan en una unidad de tiempo. Por ejemplo, si al conjunto ordenado al que aplicamos una búsqueda binaria tiene 'n' elementos, y podemos garantizar que una única búsqueda binaria puede realizarse en un tiempo unitario, entonces se requieren como mucho $\log_2 N + 1$ unidades de tiempo para devolver una respuesta.

Las medidas exactas de eficiencia son útiles para quienes verdaderamente implementan y usan algoritmos, porque tienen más precisión y así les permite saber cuánto tiempo pueden suponer que tomará la ejecución. Para algunas personas, como los desarrolladores de videojuegos, una constante oculta puede significar la diferencia entre éxito y fracaso.

A menudo se piensa que un algoritmo sencillo no es muy eficiente. Sin embargo, la sencillez es una característica muy interesante a la hora de diseñar un algoritmo, pues facilita su verificación, el estudio de su eficiencia y su mantenimiento. De ahí que muchas veces prime **la simplicidad y legibilidad del código frente a alternativas más crípticas y eficientes del algoritmo.**

Este hecho se pondrá de manifiesto en varios de los ejemplos mostrados a lo largo de este libro, en donde profundizaremos más en este compromiso.

Respecto al uso eficiente de los recursos, éste suele medirse en función de dos parámetros: **el espacio**, es decir, memoria que utiliza, y **el tiempo**, lo que tarda en ejecutarse. Ambos representan los costes que supone encontrar la solución al problema planteado mediante un algoritmo. Dichos parámetros van a servir además para comparar algoritmos entre sí, permitiendo determinar el más adecuado de entre varios que solucionan un mismo problema.

El tiempo de ejecución de un algoritmo va a depender de diversos factores como son: los datos de entrada que le suministremos, la calidad del código generado por el compilador para crear el programa objeto, la naturaleza y rapidez de las instrucciones máquina del procesador concreto que ejecute el programa, y la complejidad intrínseca del algoritmo. Hay dos estudios posibles sobre el tiempo:

1. Uno que proporciona una medida teórica (a priori), que consiste en obtener una función que acote (por arriba o por abajo) el tiempo de ejecución del algoritmo para unos valores de entrada dados.
2. Y otro que ofrece una medida real (a posteriori), consistente en medir el tiempo de ejecución del algoritmo para unos valores de entrada dados y en un ordenador concreto.

Resumiendo, el tiempo de ejecución de un algoritmo va a ser una función que mide el número de operaciones elementales que realiza el algoritmo para un tamaño de entrada dado.

Bibliografía

- “Sistemas Operativos Modernos” Andrew S. Tanenbaum. 1993. Prentice Hall.

